10.1. Introduction

Quality scenarios make the quality goals defined in Section 1.3 concrete and testable. They describe specific situations and expected system responses, allowing us to measure how well the system meets its quality attributes.

10.2. Maintainability Scenarios

- **Scenario ID:** QS-MAINT-001
- **Description:** A developer new to the Django codebase, with prior Django experience but no specific Teaching Vacancies project knowledge, can successfully implement a new, medium-complexity filter option (e.g., "contract length $>$ 6 months") for vacancy search, including model changes, form updates, view logic, and tests, within 3 developer-days.
- **Quality Goal(s) Addressed:** Maintainability
- **Stimulus:** A new feature request for an additional search filter.
- **Environment:** Development environment, access to codebase, documentation, and existing examples.
- **Response:** The new filter is implemented correctly, includes unit and integration tests, and adheres to coding standards.
- **Measure:** Time to implement (3 days), code review feedback (number of major revisions needed), test coverage for the new feature.
- **Scenario ID:** QS-MAINT-002
- **Description:** When a change is made to a core model (e.g., adding a new required field to the `Vacancy` model), the developer can identify and update all affected parts of the system (forms, views, serializers, tests) with the help of static analysis tools and the test suite, with less than 5 unexpected test failures in unrelated modules.
- **Quality Goal(s) Addressed:** Maintainability, Testability
- **Stimulus:** A requirement to add a new mandatory field to a core model.
- **Environment:** Development environment with static analysis tools (e.g., MyPy) and a comprehensive test suite.
- **Response:** All necessary code changes are made, and the system passes all tests after updates.
- **Measure:** Number of unexpected test failures in modules not directly related to the model change (target $<$ 5), time to refactor.

10.3. Testability Scenarios

- **Scenario ID:** QS-TEST-001
- **Description:** The automated unit test suite for a newly created Django app (e.g., `jobseekers`) achieves over 90% code coverage for its models and service logic.
- **Quality Goal(s) Addressed:** Testability
- **Stimulus:** Completion of development for a new Django app.
- **Environment:** CI environment with coverage reporting tools.
- **Response:** Test suite runs successfully and coverage report is generated.
- **Measure:** Unit test code coverage percentage (>= 90%).
- **Scenario ID:** QS-TEST-002
- **Description:** The full suite of automated tests (unit, integration, and critical E2E paths) can be executed in the CI pipeline within 20 minutes.
- **Quality Goal(s) Addressed:** Testability, Deployment & Operability
- **Stimulus:** A code commit to the main branch triggering the CI pipeline.
- **Environment:** CI pipeline.
- **Response:** Test suite completes, and results are reported.
- **Measure:** Total execution time of the automated test suite (20 minutes).

10.4. Scalability & Performance Scenarios

- **Scenario ID:** QS-PERF-001
- **Description:** The vacancy search results page (for a common search query) loads and becomes interactive within 2 seconds (95th percentile) under a simulated load of 500 concurrent users performing diverse search operations.
- **Quality Goal(s) Addressed:** Performance, Scalability
- **Stimulus:** Load test simulating 500 concurrent users searching for vacancies.
- **Environment:** Staging environment with production-like data volume and load testing tools (e.g., Locust, k6).
- **Response:** Page load times and server-side response times are recorded.
- **Measure:** 95th percentile page load time (2 seconds), server error rate (< 0.1%).

- **Scenario ID:** QS-PERF-002
- **Description:** The system can process the import of 10,000 new vacancies from an external ATS feed via the API within 1 hour, with vacancy data being available in search results within 5 minutes of successful import and processing of each vacancy.
- **Quality Goal(s) Addressed:** Performance, Scalability
- **Stimulus:** API submission of a batch of 10,000 vacancies.
- **Environment:** Staging or performance testing environment.
- **Response:** Vacancies are created, processed (e.g., search index updated), and become searchable.
- **Measure:** Total time to import the batch (1 hour), time from individual vacancy import to search availability (5 minutes).

10.5. Security Scenarios

- **Scenario ID:** QS-SEC-001
- **Description:** An external penetration test conducted by a DfE-approved third party identifies no critical or high-severity vulnerabilities related to OWASP Top 10 (e.g., XSS, SQL Injection, Broken Access Control) in the job application submission and review process.
- **Quality Goal(s) Addressed:** Security
- **Stimulus:** Scheduled penetration test.
- **Environment:** Pre-production or production-like environment.
- **Response:** Penetration test report.
- **Measure:** Number of critical/high vulnerabilities identified (target: 0). Time to remediate any identified medium/low vulnerabilities.
- **Scenario ID:** QS-SEC-002
- **Description:** All sensitive Personally Identifiable Information (PII) within the `User` model (e.g., `family_name`, `given_name`) and `JobApplication` model is confirmed to be encrypted at rest in the database.
- **Quality Goal(s) Addressed:** Security, Data Integrity
- **Stimulus:** Database audit/inspection.
- **Environment:** Development or staging database.
- **Response:** Database records are inspected.

- **Measure:** Confirmation that specified fields are stored as ciphertext and can be decrypted correctly by the application.

10.6. Interoperability & Extensibility Scenarios

- **Scenario ID:** QS-INTEROP-001
- **Description:** A third-party developer, using the provided API documentation, can successfully build a client application that posts a new vacancy via the ATS API endpoint within 2 developer-days.
- **Quality Goal(s) Addressed:** Interoperability, Extensibility
- **Stimulus:** Tasking an internal or external developer to integrate with the ATS API.
- **Environment:** Development/sandbox environment with API access and documentation.
- **Response:** Successful vacancy creation via the API.
- **Measure:** Time taken (2 days), number of support requests needed from the test developer (target low).
- **Scenario ID:** QS-EXTEND-001
- **Description:** Adding a new, simple read-only field to an existing API endpoint (e.g., adding `organisation.phase` to a vacancy API response) requires less than 0.5 developer-days, including changes to serializers and documentation.
- **Quality Goal(s) Addressed:** Extensibility, Maintainability
- **Stimulus:** Feature request to expose an additional existing data field via an API.
- **Environment:** Development environment.
- **Response:** API endpoint updated and documentation regenerated.
- **Measure:** Time taken (0.5 days).

10.7. Deployment & Operability Scenarios

- **Scenario ID:** QS-DEPLOY-001
- **Description:** A new code release (including database migrations if any) can be deployed to the production environment via the automated CI/CD pipeline within 30 minutes from the point of merging to the main branch, with zero perceived downtime for users.
- **Quality Goal(s) Addressed:** Deployment & Operability

- **Stimulus:** Merge of a pull request to the main branch.
- **Environment:** Production environment, CI/CD system.
- **Response:** Successful deployment, application remains available.
- **Measure:** Deployment time (30 minutes), user-perceived downtime (target: 0 seconds).
- **Scenario ID:** QS-OPER-001
- **Description:** In the event of a critical application error in production, the monitoring system generates an alert to the support team within 5 minutes, and the logging system provides sufficient contextual information (e.g., stack trace, request details) for a developer to begin diagnosing the issue within 15 minutes of the alert.
- **Quality Goal(s) Addressed:** Operability
- **Stimulus:** A simulated or actual critical error in production.
- **Environment:** Production environment with monitoring and logging systems.
- **Response:** Alert received, logs available.
- **Measure:** Time to alert (5 minutes), availability and clarity of logs for diagnosis.

10.8. Accessibility Scenarios

- **Scenario ID:** QS-ACCESS-001
- **Description:** Key user journeys (job search, view vacancy, start application, publisher creates vacancy) pass automated accessibility checks (e.g., Axe tool) with no WCAG 2.1 A or AA violations.
- **Quality Goal(s) Addressed:** Accessibility
- **Stimulus:** Running automated accessibility tests against deployed frontend pages.
- **Environment:** CI pipeline or staging environment.
- **Response:** Accessibility test report.
- **Measure:** Number of WCAG 2.1 A/AA violations (target: 0).
- **Scenario ID:** QS-ACCESS-002
- **Description:** A manual accessibility audit conducted by an accessibility specialist on the core jobseeker and publisher user journeys confirms compliance with WCAG 2.1 Level AA, with a report detailing any identified issues.

- **Quality Goal(s) Addressed:** Accessibility
- **Stimulus:** Scheduled manual accessibility audit.
- **Environment:** Pre-production or production environment.
- **Response:** Accessibility audit report.
- **Measure:** Confirmation of compliance; number and severity of any identified issues, and a plan for remediation.

10.9. Data Integrity & Migration Scenarios

- **Scenario ID:** QS-DATA-001
- **Description:** After the full data migration from the legacy Rails system to the new Django system, automated validation scripts comparing key counts and checksums for critical entities (Users, Organisations, Vacancies, Job Applications, Subscriptions) show less than 0.01% discrepancy.
- **Quality Goal(s) Addressed:** Data Integrity & Migration
- **Stimulus:** Completion of the data migration process.
- **Environment:** Post-migration production database.
- **Response:** Validation script output.
- **Measure:** Discrepancy rate (target $< 0.01\%$), successful validation of data relationships.
- **Scenario ID:** QS-DATA-002
- **Description:** A sample of 100 complex legacy vacancy records, including those with associated applications, documents, and varied field data, are manually verified to have been migrated to the new Django system with all data fields accurately transformed and all relationships intact.
- **Quality Goal(s) Addressed:** Data Integrity & Migration
- **Stimulus:** Completion of the data migration process.
- **Environment:** Post-migration production database, access to legacy data for comparison.
- **Response:** Manual verification checklist for each sampled record.
- **Measure:** Percentage of sampled records migrated accurately (target: 100%). Any identified issues are categorized and a plan for correction is made.